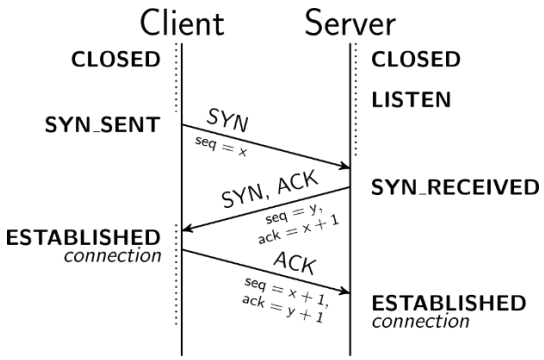
Group Name: Go With the Flow
Jacob Lebkuecher– ID: 904026388
Ethan Van Brunt – ID: 904031060
Ryan Giacoboni – ID: 904003021

What is it?

Transmission Control Protocol (TCP) is one of the fundamental transmission protocols supporting communication over the internet. Unlike the User Datagram Protocol (UDP), TCP ensures that data reaches its destination and in correct order, regardless of speed or network reliability.

Connection Establishment

Before communication can occur, a device must establish a connection to another device it wishes to communicate with. To establish a connection, TCP utilizes a *Three-Way Handshake*.



1. The sender sends a SYN packet to the receiver, requesting a connection.
2. Once received, the receiver sends a SYN-ACK packet, acknowledging the request and agreeing (or disagreeing) to establish the connection.
3. Upon receiving the SYN-ACK, the sender responds with an ACK packet, confirming the connection has been established.

Segmentation

Once connected, both devices can start sending data over the network. To do this reliably, however, requires TCP to segment the data into chunks. The protocol adds a TCP header to each chunk. It contains necessary information the chunk needs in order to be sent correctly, including the source port, destination port, sequence number, and more.

TCP header format[19]

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Sequence Number																															
8	64	Acknowledgement Number (meaningful when ACK bit set)																															
12	96	Data Offset				Reserved				CWR	ECE	URG	ACK	PSH	RST	SYN	FIN	Window															
16	128	Checksum																Urgent Pointer (meaningful when URG bit set)[20]															
20	160	(Options) If present, Data Offset will be greater than 5. Padded with zeroes to a multiple of 32 bits, since Data Offset counts words of 4 octets.																															
⋮	⋮																																
56	448																																
60	480	Data																															
64	512																																
⋮	⋮																																

Group Name: Go With the Flow
Jacob Lebkuecher– ID: 904026388
Ethan Van Brunt – ID: 904031060
Ryan Giacoboni – ID: 904003021

Sequencing

Within the TCP header of each packet lies a sequence number identifying each byte of data sent. This number is included so the data can be reconstructed by the receiver in order, even if the data arrives out of order.

Acknowledgement

The receiver, for each packet (or group of packets) it receives, responds with an ACK to the sender. This ACK confirms the receiver has received everything up to a specific byte, ensuring the sender knows that data is being sent correctly.

Retransmission

To ensure no loss of data, TCP must send every single packet, including packets that were lost. To do this, TCP initializes a timer with an estimate of the ACK's arrival time. If the sender does not receive an ACK within the estimated timer, it assumes the packet was lost and resends it.

Flow Control

Due to the nature of TCP, packets are sent whenever they are ready. An influx of packets can be too much for the receiver to handle. The receiver may not be able to process the incoming packet data at the same rate, causing packet loss and a severe drop in performance.

TCP prevents this scenario by using a sliding window. In each ACK, the receiver advertises a window size (called the *receive window*), indicating the amount (in bytes) of buffer space available. The sender can only send data up to the advertised window size.

Due to the nature of connections, the rate of data flow to the receiver is always changing. TCP's flow control handles this by dynamically changing the window size. As the receiver processes data and buffer space frees up, the window size increases. Conversely, as the receiver gets backed up, the window size decreases.

Congestion Control

If too many senders flood a network with packets all at once, the router's buffer can overflow, leading to network congestion followed by widespread packet loss and a severe drop in performance.

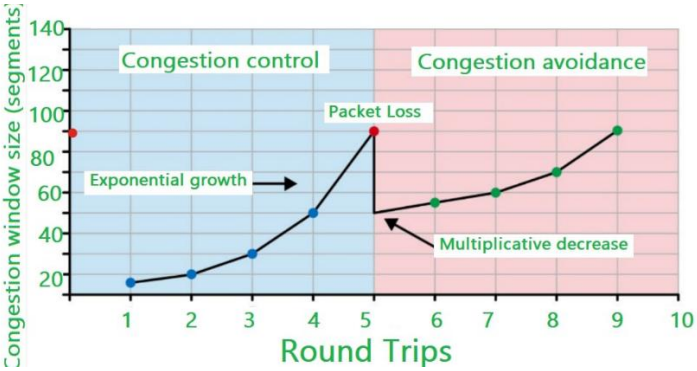
TCP prevents this exact scenario by providing the sender with a *congestion window*. This window limits how much data can be in flight. It works alongside the receive window to keep the data flow below a rate that would trigger network congestion.

TCP controls the congestion window using AIMD (Additive Increase, Multiplicative, Decrease).

AIMD has two phases, the first is *Slow Start*. Every round trip time, TCP probes for available bandwidth and increases the window size exponentially, doubling each round-trip time, until reaching a threshold or data loss is detected.

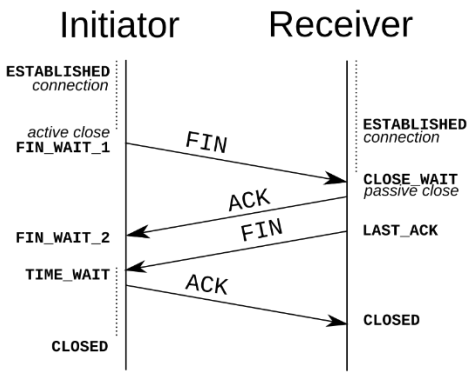
Once the congestion window reaches a threshold, AIMD moves into *Congestion Avoidance*, that is, the window size grows linearly, increasing by 1 each round-trip time.

When packet loss occurs due to duplicate ACKs, the congestion window size is cut in half, and TCP continues with Congestion Avoidance. If packet loss is due to timeout, then the congestion window is set back to 1 and TCP enters Slow Start.



Connection Termination

To close a TCP connection, both devices must coordinate with each other to ensure no data is lost. To terminate a connection, TCP utilizes a *Four-Way Handshake*.



1. The sender wishing to close the connection sends a FIN packet to the receiver, indicating the device wishes to end the connection.
2. Upon receiving the FIN, the receiver sends an ACK packet, acknowledging the request to end the connection.
3. The receiver sends its own FIN once it is ready to end the connection.
4. Once received, the sender responds with an ACK, acknowledging the receiver.